Git Workshop

Day 1 — Foundations | Day 2 — Collaboration & Advanced

ReDI School of Digital Integration — Aarhus

Day 1 — Agenda (4 hours)

🎯 *Day 1 = Personal Git — work locally with confidence*

10:00 – 10:30	**Setup & Introduction**
10:30 – 11:05	**Core Workflow (add → commit → status)**
11:05 – 11:35	**Exploring History (log, diff, blame)**
11:35 – 12:05	**Branching Basics**
12:05 – 12:25	Lunch Break 🍴
12:25 – 13:10	**Going Back to a Previous Commit**
13:10 – 13:40	**Branching & Merging Deep Dive**
13:40 – 14:00	**Day 1 Wrap-up & Preview Day 2**

Workshop Goals

- Understand Git fundamentals — what it does and why it matters
- Gain hands-on experience — every concept comes with an exercise
- Learn how to undo mistakes safely — rollback without fear
- Master collaboration workflows — remotes, PRs, conflict resolution
- Level up with advanced techniques — cherry-pick, squash, rebase
- Please ask questions — there are no silly questions!

Terminal Guide for Windows Users

Git Bash

RECOMMENDED

Installed with Git for Windows

Unix-like commands (ls, cd, cat)

Closest to macOS/Linux terminals

Best for following this workshop

PowerShell

WORKS BUT DIFFERENT

Pre-installed on Windows

Different commands (dir, cls, etc.)

Some Git commands behave differently

Use if you prefer native Windows

VS Code Terminal

EASIEST SETUP

Built into VS Code (Ctrl + `)

Can select Git Bash as default shell

Integrated with your project folder

Set via: Terminal > Default Profile

💡 Tip: In VS Code, open Settings → search "terminal default profile" → select Git Bash

Common Windows Issues & Fixes

`'git' is not recognized as a command`

→ Restart terminal after installing Git, or add Git to your PATH manually

`Line ending warnings (CRLF vs LF)`

→ git config --global core.autocrlf true

`Permission denied (SSH key)`

→ Use HTTPS clone URL instead, or generate SSH key with ssh-keygen

`Terminal shows weird characters or colors`

→ Use Git Bash or VS Code terminal instead of CMD

`Can't see .git folder`

→ Enable "Show hidden files" in Windows Explorer settings

Setup & Introduction

What Git is, why it matters, and getting ready

The Problem Without Version Control

- ✗ Hard to track who changed what and when
- ✗ Easy to overwrite each other's work
- ✗ Rolling back is painful and error-prone
- ✗ Filenames like: report_final_v2_FINAL_fixed(2).docx

> 💬 Ask: "Who has worked on a project with filenames like this?"

What Is Git?

Git is a free and open-source distributed version control system designed to handle everything from small to very large projects.

Tracks Snapshots	Every commit is a full snapshot of your files, not a diff
Distributed	Every clone is a complete repository with full history
Branching & Merging	Lightweight branches make parallel work easy

Core Git Concepts

Repository (repo)	A folder tracked by Git — contains a hidden .git directory
Commit	A snapshot of changes with a message, author, and timestamp
Staging Area	Where you prepare changes before committing them
Branch	An independent line of development
Remote	A shared repository (e.g. GitHub, GitLab)

```
Working Directory  →  Staging Area  →  Repository (.git)
```

Setup Instructions

1 INSTALL

```
git-scm.com/downloads
macOS: brew install git
Windows: Git for Windows installer
```

2 VERIFY

```
git --version
```

3 CONFIGURE

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
```

4 CHECK

```
git config --list
```

Core Workflow

Edit → Stage → Commit

The Git Workflow: Edit → Stage → Commit

EDIT	STAGE	COMMIT
Make changes to your files	Choose which changes to include	Save a snapshot with a message

> Git doesn't auto-track changes — you decide what gets staged and committed.

`git status` → check state `git add <file>` → stage `git commit -m "msg"` → save

Creating a Repository

1. **Create a folder and enter it**

```
mkdir my-first-repo
cd my-first-repo
```

2. **Initialize Git**

```
git init
```

3. **Verify the .git folder**

```
ls -a          # macOS / Git Bash
dir /a         # Windows CMD
```

Adding, Staging & Committing

```
# Check what's changed
git status

# Create a file (or edit one)
echo "Hello Git!" > hello.txt

# Stage the file
git add hello.txt
git add .                    # stage ALL changes

# Commit with a message
git commit -m "Add hello.txt"

# View your commit
git log --oneline
```

.gitignore — Keeping Your Repo Clean

Tell Git which files to ignore (never track).

```
.gitignore

# Dependencies
node_modules/

# IDE files
.vscode/
.idea/

# OS files
.DS_Store
Thumbs.db

# Environment
.env
```

Common patterns:

`*.log`	Ignore all .log files
`build/`	Ignore entire directory
`!important.log`	Exception — do track this
`**/*.tmp`	Ignore in all subdirs

📚 Templates: github.com/github/gitignore — ready-made for most languages

Exercise — Core Workflow

1. Create a new repository

```
mkdir git-exercise && cd git-exercise && git init
```

2. Create a file, stage it, and commit

```
echo "Hello" > hello.txt && git add hello.txt && git commit -m "First commit"
```

3. Create 2 more commits with different files

4. Create a commit with message: "find me"

5. Create a .gitignore that ignores .log files

Exploring History

Understanding commits, diffs, and blame

Commit Identifiers (SHA-1 Hash)

Every commit has a unique 40-character hash. You usually only need the first 7 characters.

```
8d1922ecba76596c15f583abb4896110139420ef
```

Short hash (7 chars) — usually enough

Special references:

`HEAD`	Current commit you're on
`HEAD~1`	One commit before HEAD
`HEAD~3`	Three commits before HEAD
`main`	Tip of the main branch

Viewing History — Key Commands

Press q to exit the log viewer

`git log`	See full commit history (author, date, message)
`git log --oneline`	Compact view — one line per commit
`git log --graph --decorate --all`	Visual branch graph
`git log --author="name"`	Filter by author
`git log --grep="keyword"`	Search commit messages
`git show <commit>`	Show details + diff of a specific commit
`git diff <c1> <c2>`	Compare two commits
`git blame <file>`	Who changed each line and when

Exercise — Exploring History

1. Find the commit with message "find me"

```
git log --grep="find me"
```

2. Compare your first and last commit

```
git diff <first-hash> <last-hash>
```

3. Use git blame on one of your files

```
git blame hello.txt
```

4. Try the visual log format

```
git log --oneline --graph --all
```

Branching Basics

Work on features in isolation

What Is a Branch?

A branch is an independent line of development. It lets you work on features without affecting the main code.

Isolation

Work on features without breaking main

Collaboration

Multiple people work in parallel

Experimentation

Try ideas without risk

Creating & Switching Branches

> HEAD is a pointer to the current branch/commit you are on.

`git branch`	List all branches	INFO
`git branch feature-x`	Create a new branch	CREATE
`git checkout feature-x`	Switch to a branch	SWITCH
`git checkout -b feature-x`	Create + switch in one step	SHORTCUT
`git switch feature-x`	Modern way to switch (Git 2.23+)	SWITCH
`git switch -c feature-x`	Modern create + switch	SHORTCUT

Merging a Branch

```
# Step 1: Switch to the branch you want to merge INTO
git checkout main

# Step 2: Merge the feature branch
git merge feature-x

# Step 3 (optional): Delete the merged branch
git branch -d feature-x
```

> Always switch to the receiving branch (main) before merging.

Before merge: `main: A ── B` `feature: A ── B ── C ── D`

After merge: `main: A ── B ── C ── D  (HEAD)`

Exercise — Branching & Merging

1. Create a feature branch called "my-feature"

```
git checkout -b my-feature
```

2. Add a new file and commit it on the branch

3. Switch back to main

```
git checkout main
```

4. Merge your feature branch into main

```
git merge my-feature
```

5. Delete the merged branch

```
git branch -d my-feature
```

Lunch Break

12:05 – 12:25

Going Back to a Previous Commit

Checkout, revert, reset — when to use what

git checkout <commit> — Time Travel

```
git checkout abc1234
```

WHAT IT DOES

Moves your working directory to a specific commit without changing any branch.

USE CASE

Exploring an old commit, testing code at that point in time.

⚠️ DETACHED HEAD

You are NOT on a branch. New commits here won't belong to any branch unless you create one.

```
To save changes: git switch -c new-branch   or   git checkout -b new-branch
```

git revert <commit> — Safe Undo

```
git revert abc1234
```

WHAT IT DOES	Creates a NEW commit that undoes the changes from a previous commit.
USE CASE	Undo a commit on a shared/pushed branch safely.
KEY BENEFIT	Does NOT rewrite history — safe for shared branches. Everyone can see what was undone and why.

✅ SAFE for shared branches — this is the preferred way to undo on public repos

git reset — Rewrite Local History

```
git reset --soft|--mixed|--hard <commit>
```

Flag	Branch Pointer	Staging Area	Working Dir
`--soft`	Moved	✅ Kept	✅ Kept
`--mixed`	Moved	❌ Unstaged	✅ Kept
`--hard`	Moved	❌ Deleted	❌ Deleted

⚠️ --hard is DESTRUCTIVE — lost changes cannot be recovered (easily). Never use on shared branches!

Checkout vs Revert vs Reset — When to Use What

	checkout	revert	reset
Changes history?	No	No (adds commit)	Yes (rewrites)
Safe for shared?	Yes	Yes ✅	No ⚠️
Creates new commit?	No	Yes	No
Best for...	Exploring	Undoing on shared	Undoing locally

Quick Decision Guide:

```
Just exploring?                    → git checkout <commit>
Undo a commit on a shared branch? → git revert <commit>
Undo local commits (not pushed)?  → git reset --soft/--mixed <commit>
Nuclear option (lose everything)?  → git reset --hard <commit>
```

Exercise — Rollback Scenarios

1. Revert your last commit and check the log

```
git revert HEAD && git log --oneline
```

2. Use git reflog to find the reverted commit

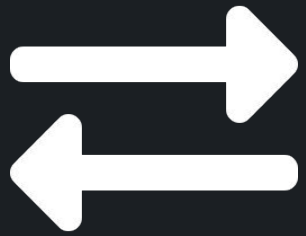
```
git reflog
```

3. Create a branch from the old commit

```
git checkout -b recovered <hash>
```

4. Merge the recovered branch back into main

5. Try git reset --soft HEAD~1 and check git status



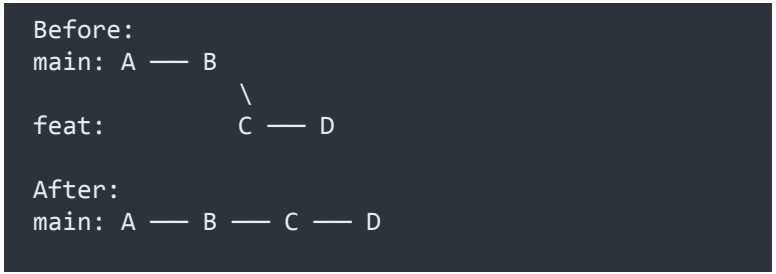
Branching & Merging Deep Dive

Fast-forward vs non-fast-forward

Fast-Forward vs Non-Fast-Forward Merge

Fast-Forward Merge

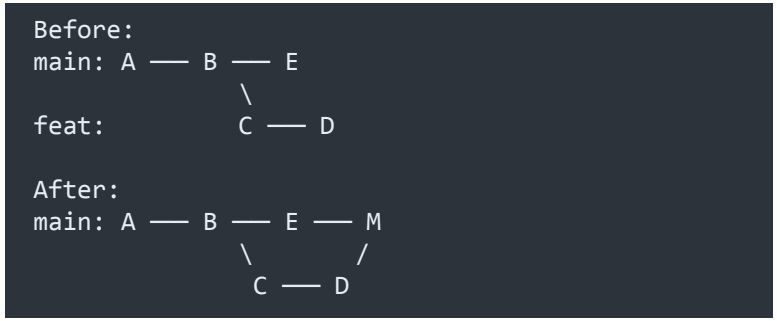
When main has NO new commits since you branched.



No merge commit created.
Git just moves the pointer forward.

Non-Fast-Forward Merge

When BOTH branches have new commits.



Creates a merge commit (M).
Press :wq to exit the editor.

Day 1 Wrap-up — What You Learned

Setup & Config	`git init, git config`
Core Workflow	`add → commit → status`
History	`log, show, diff, blame`
Branching	`branch, checkout, merge`
Rollback	`checkout, revert, reset`

Tomorrow: Collaboration — conflicts, remotes, PRs + Advanced: cherry-pick, squash, rebase!

Git Workshop

Day 2 — Collaboration, Advanced Techniques & Real-World Git

ReDI School of Digital Integration — Aarhus

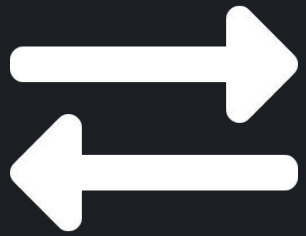
Day 2 — Agenda (4 hours)

🎯 *Day 2 = Team Git + Advanced — collaborate and level up*

10:00 – 10:15	Review & Warm-up
10:15 – 10:55	**Resolving Conflicts**
10:55 – 11:50	**Remote Repositories (clone, push, pull)**
11:50 – 12:10	Lunch Break 🍴
12:10 – 12:45	**Collaboration Workflows & PRs**
12:45 – 13:10	**Undoing Mistakes Safely (stash, reflog)**
13:10 – 13:40	**Advanced Techniques (cherry-pick, squash, rebase)**
13:40 – 13:50	**Git GUIs & IDE Integration**
13:50 – 14:00	**Best Practices, Cheat Sheet & Wrap-up**

Quick Review — Day 1 Recap

- ? What are the 3 stages of the Git workflow?
- ? What command creates a new branch?
- ? What's the difference between revert and reset?
- ? What does HEAD point to?
- ? How do you see commit history?



Resolving Conflicts

When Git can't merge automatically

What Is a Merge Conflict?

A conflict occurs when two branches modify the same lines in the same file. Git doesn't know which version to keep, so it asks you to decide.

Common triggers:

→ Two people edit the same line in the same file

→ One person deletes a file another person edited

→ Divergent branches with overlapping changes

What conflict markers look like:

```
<<<<<<< HEAD
My changes on this branch
=======
Their changes on the other branch
>>>>>>> feature-branch
```

Resolving Conflicts — Step by Step

> 💡 VS Code highlights conflicts with colors and offers Accept Current / Accept Incoming / Accept Both buttons.

1 **Git tells you there's a conflict**

```
git merge feature-x
# CONFLICT (content): Merge conflict in file.txt
```

2 **Open the file and edit — decide what to keep**

```
# Remove the markers <<<<<<< ======= >>>>>>>
# Keep the code you want
```

3 **Stage the resolved file**

```
git add file.txt
```

4 **Commit to finalize**

```
git commit
# Git auto-creates a merge commit message
```

Exercise — Resolving Conflicts

1. Create a branch and modify line 1 of hello.txt
2. Switch to main and modify the SAME line 1 differently
3. Try to merge the branch — observe the conflict

```
git merge <branch-name>
```

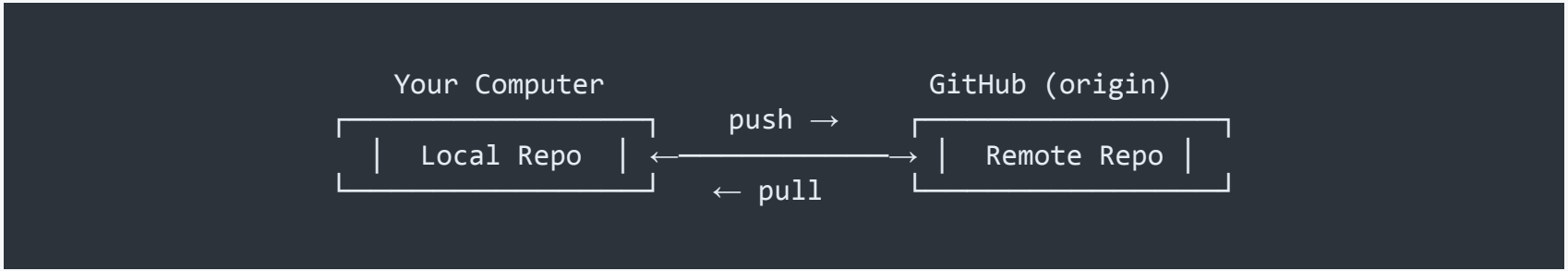
4. Open the file, resolve the conflict manually, then add + commit
5. Verify with git log --oneline --graph

Remote Repositories

Connect your local repo to the world

What Are Remote Repositories?

A remote is a version of your repository hosted on a server (e.g. GitHub, GitLab). It's how you share code and collaborate.



`git remote -v`	List remotes and their URLs
`git remote add origin <url>`	Connect local repo to a remote

> "origin" is the default name for your main remote — it's just a convention.

GitHub Setup

1

CREATE GITHUB ACCOUNT

Visit github.com → Sign Up

2

SIGN IN VIA VS CODE

Account icon → Backup & Sync → Sign in with GitHub

3

CREATE A REMOTE REPO

GitHub → + New repository → name it → Create

4

CONNECT LOCAL REPO

git remote add origin https://github.com/you/repo.git

The Remote Commands

`git clone <url>`

Download a remote repo (creates folder with full history)

`git push origin main`

Upload your commits to the remote

`git pull origin main`

Download + merge remote changes

`git fetch`

Download remote changes WITHOUT merging

`git push -u origin feature-x`

Push a new branch and set tracking

Authentication — HTTPS vs SSH

HTTPS

Easier to set up

Uses username + personal access token

Token generated at GitHub Settings → Developer → Tokens

Git remembers credentials with:
git config --global credential.helper store

Works through firewalls

SSH

More secure, no re-login

Uses SSH key pair (public + private)

Generate: ssh-keygen -t ed25519

Add public key to GitHub → Settings → SSH Keys

Test: ssh -T git@github.com

💡 For this workshop, HTTPS is fine. VS Code handles auth via the GitHub extension.

Exercise — Remote Repositories

1. Create a new repository on GitHub

2. Connect your local repo to the remote

```
git remote add origin <url>
```

3. Push your local commits

```
git push -u origin main
```

4. Ask a partner to clone your repo

```
git clone <url>
```

5. Partner makes a change, pushes — then you pull

```
git pull origin main
```

Lunch Break

12:00 – 12:20

Collaboration Workflows

How teams work together with Git

Common Collaboration Workflows

Feature Branch Workflow	Each feature in its own branch. Merge to main via PR after review. Most common for teams.
Forking Workflow	Each developer forks the repo. Contribute via PRs from your fork. Common in open source.
Gitflow	Structured branches: main, develop, feature, release, hotfix. Best for scheduled releases.
Trunk-Based	Short-lived branches, frequent merges to main. Requires strong CI/CD. Used by large teams.

Pull Requests (PRs) — The Collaboration Hub

A Pull Request is a way to propose changes and request code review before merging into main.

1 Create a branch and push it

```
git checkout -b feature-x
# ... make changes, commit ...
git push -u origin feature-x
```

2 Open a PR on GitHub

```
Go to your repo → Compare & Pull Request
Add title, description, reviewers
```

3 Review & Discuss

```
Team reviews code, leaves comments
Make fixes if needed, push again
```

4 Merge

```
Click 'Merge pull request' on GitHub
Delete the branch
```

Exercise — Pull Request Workflow

1. Create a feature branch and make changes
2. Push the branch to GitHub

```
git push -u origin my-feature
```

3. Open a Pull Request on GitHub
4. Ask a partner to review and comment on your PR
5. Merge the PR and pull the changes locally

```
git checkout main && git pull
```

Undoing Mistakes Safely

Stash, reflog, and disaster recovery

git stash — Save Work Without Committing

```
# Save current changes to the stash
git stash

# List all stashes
git stash list

# Apply the last stash (keep it in the list)
git stash apply

# Apply and remove from list
git stash pop
```

When to use stash:

→ Need to switch branches but have uncommitted work

→ Want to test something quickly without losing progress

→ Pull from remote but have local changes that conflict

git reflog — Your Safety Net

reflog records EVERY movement of HEAD. Even after reset --hard, your commits are still there for ~90 days.

```
# See the reflog
git reflog

# Example output:
abc1234 HEAD@{0}: reset: moving to HEAD~2
def5678 HEAD@{1}: commit: Add login feature
ghi9012 HEAD@{2}: commit: Fix bug in navbar
```

Recovery scenarios:

Accidentally did git reset --hard	git reflog → find the commit → git reset --hard <hash>
Deleted a branch with unmerged work	git reflog → find last commit → git checkout -b recovered <hash>

Best Practices

Write better commits, work better as a team

Commit Message Guidelines

✅ GOOD

```
feat: add lazy load for images

fix: prevent form double submit

docs: update API documentation

chore: upgrade dependencies

refactor: extract auth middleware
```

❌ BAD

```
fixed stuff

update

oops

WIP

I think this works now?
```

Use imperative mood: "Add feature" not "Added feature" • Keep subject < 50 chars • Capitalize first word

Team Conventions & Best Practices

Always use Pull Requests	For visibility, review, and a clean merge history
Agree on a branching model	Feature Branch, Gitflow, or Trunk-Based — don't mix
Resolve conflicts locally	Better control than resolving in GitHub's UI
Keep branches small	Easier to review, less likely to conflict
Delete merged branches	Keep the repo clean — old branches cause confusion
Use .gitignore from day one	Never commit node_modules, .env, or IDE files

📚 Workshop repo: github.com/ReDI-Aarhus/Git-GitHub-Workshop

Advanced Git Techniques

Cherry-pick, squash, rebase — levelling up your workflow

git cherry-pick — Grab a Single Commit

What is it?

Apply a specific commit from one branch onto another — without merging the entire branch.

When to use it?

- Hotfix from a feature branch → main
- Port a bug fix across release branches
- Pull one useful commit without merging all

```
# Find the commit hash you want
$ git log --oneline feature-branch
a1b2c3d  Add input validation
e4f5g6h  Fix login bug

# Switch to the target branch
$ git checkout main

# Cherry-pick the specific commit
$ git cherry-pick a1b2c3d
```

⚠️ Cherry-pick creates a NEW commit with a different hash — the original stays on its branch

Squashing Commits — Clean Up Your History

BEFORE — messy

```
f1a2b3c  WIP

d4e5f6a  fix typo

b7c8d9e  oops forgot file

a0b1c2d  actually done now

e3f4a5b  final fix I promise
```

AFTER — squashed ✨

```
abc123d  feat: add user login form
```

5 messy commits ⟶ 1 clean commit
with a clear, descriptive message

```
# Method 1: Interactive rebase (squash last 5 commits)
$ git rebase -i HEAD~5
# In editor: change 'pick' to 'squash' (or 's') for
# all commits except the first one

# Method 2: Squash merge (when merging a branch)
$ git merge --squash feature-branch
$ git commit -m "feat: add user login form"
```

git rebase — Rewriting History for a Clean Timeline

git merge

- Keeps full branching history
- Creates a merge commit
- Non-destructive — safe for shared branches
- History can look cluttered

git rebase

- Re-applies your commits on top of base
- Produces a linear, clean history
- Rewrites commit hashes!
- Never rebase shared/public branches

```
# Update your feature branch with latest main
$ git checkout feature-branch
$ git rebase main

# If conflicts occur, fix them then:
$ git add .
$ git rebase --continue

# Abort if things go wrong:
$ git rebase --abort
```

🔴 THE GOLDEN RULE

Never rebase commits that have been pushed and shared with others.

Rebase rewrites history — if others have based work on those commits, it will cause conflicts and confusion.

Rebase Workflow — Keeping Feature Branches Up to Date

1. **Start on your feature branch**

```
$ git checkout feature-login
```

2. **Fetch latest changes from remote**

```
$ git fetch origin
```

3. **Rebase onto updated main**

```
$ git rebase origin/main
```

4. **Resolve any conflicts (if needed)**

```
$ git add . && git rebase --continue
```

5. **Force push your updated branch**

```
$ git push --force-with-lease
```

> 💡 Use --force-with-lease instead of --force — it won't overwrite others' work

Git GUIs & IDE Integration

Standalone GUIs

GitHub Desktop
Simple, clean — great for beginners

GitKraken
Visual branch graph, drag & drop

Sourcetree
Free, feature-rich (Atlassian)

Fork
Fast, lightweight (macOS / Win)

IDE Integration

VS Code
Built-in Git + GitLens extension

JetBrains IDEs
Powerful Git tools, visual diff

Vim / Neovim
fugitive.vim, lazygit

Sublime Merge
Dedicated Git client from Sublime

Terminal-Based

lazygit
Fast TUI — keyboard-driven

tig
Ncurses log viewer & browser

git log --graph
Built-in ASCII branch view

delta / diff-so-fancy
Beautiful diffs in terminal

💡 Learn the CLI first — GUIs are great companions, but knowing the commands gives you full control

Git Command Cheat Sheet

BASICS	HISTORY	BRANCHES	REMOTE	UNDO	ADVANCED
`git init` Start repo	`git log --oneline` Compact log	`git checkout -b name` New branch	`git clone <url>` Download	`git revert <hash>` Safe undo	`git cherry-pick <h>` Grab commit
`git add .` Stage all	`git diff` See changes	`git merge name` Merge	`git push` Upload	`git reset --soft` Undo commit	`git rebase main` Replay on top
`git commit -m "msg"` Commit	`git show <hash>` Commit details	`git branch -d name` Delete	`git pull` Download+merge	`git stash` Save WIP	`git merge --squash` Squash merge
`git status` Check state	`git blame <file>` Line history	`git switch name` Switch	`git fetch` Download only	`git reflog` Safety net	`git rebase -i HEAD~n` Interactive

Thank You!

You now have the tools to use Git with confidence

Day 1: Personal Git — init, commit, branch, rollback
Day 2: Team Git — remotes, conflicts, PRs, recovery
Advanced: cherry-pick, squash, rebase, GUIs

`github.com/ReDI-Aarhus/Git-GitHub-Workshop`